Explain RAG simply

Saved from ChatGPT on 11/7/2026 - 60 messages

YOU

Question 1: Explain RAG simply.
Please keep it short.

CHATGPT

Answer 1: RAG in one minute

RAG = *retrieval* + generation. Steps:

1. Split documents into chunks.
2. Find chunks that match the question.
3. Give them to the model with the question.

Tiny Python version:

```
def retrieve(query, docs):
    """Return the docs that share a word with the query."""
    words = set(query.lower().split())
    results = []
    for doc in docs:
        if words & set(doc.lower().split()):
            results.append(doc)


    return results


docs = ["RAG uses retrieval", "Pizza is tasty", "retrieval helps <LLMs> & search"]
found = retrieve("Retrieval", docs)
assert len(found) == 2, found
print("OK, found:", found)
```

Run it with `python rag.py`.

YOU

Question 2: Compare search methods in a table.

CHATGPT

Answer 2: Comparison table

Here is how the options compare:

Method	Speed	Needs GPU?	Example
Keyword search	Fast	No	`grep "rag"`
Vector search	Medium	Sometimes	a | b pipes
Re-ranking	Slow	Yes	cross-encoder

Pick keyword search for small projects.

YOU

Question 3: Give me a study plan.
Please keep it short.

CHATGPT

Answer 3: Study plan

- Week 1
  - Python basics
  - Lists & dictionaries
- Week 2
  - APIs
  - Embeddings

3. Sort some scores in JavaScript:

```
const scores = [42, 7, 19];
const sorted = [...scores].sort((a, b) => a - b);
if (sorted.join(",") !== "7,19,42") throw new Error("bad sort");
console.log("OK, sorted:", sorted.join(" < "));
```

4. Check the output says `OK`.

YOU

Question 4: What are the key ideas?

CHATGPT

Answer 4: Key ideas

> Good retrieval matters more than a bigger model.

Cosine similarity is `\cos(\theta) = \frac{a \cdot b}{|a||b|}`. Read the Wikipedia page.

Use `top_k = 3` to start.

Quick check: what does RAG add?

○ Retrieval

○ Bigger GPUs

YOU

Question 5: Explain RAG simply.
Please keep it short.

CHATGPT

Answer 5: RAG in one minute

RAG = *retrieval* + generation. Steps:

1. Split documents into chunks.
2. Find chunks that match the question.
3. Give them to the model with the question.

Tiny Python version:

```
def retrieve(query, docs):
    """Return the docs that share a word with the query."""
    words = set(query.lower().split())
    results = []
    for doc in docs:
        if words & set(doc.lower().split()):
            results.append(doc)


    return results


docs = ["RAG uses retrieval", "Pizza is tasty", "retrieval helps <LLMs> & search"]
found = retrieve("Retrieval", docs)
assert len(found) == 2, found
print("OK, found:", found)
```

Run it with `python rag.py`.

YOU

Question 6: Compare search methods in a table.

CHATGPT

Answer 6: Comparison table

Here is how the options compare:

Method	Speed	Needs GPU?	Example
Keyword search	Fast	No	`grep "rag"`
Vector search	Medium	Sometimes	a | b pipes
Re-ranking	Slow	Yes	cross-encoder

Pick keyword search for small projects.

YOU

Question 7: Give me a study plan.
Please keep it short.

CHATGPT

Answer 7: Study plan

- Week 1
  - Python basics
  - Lists & dictionaries
- Week 2
  - APIs
  - Embeddings

3. Sort some scores in JavaScript:

```
const scores = [42, 7, 19];
const sorted = [...scores].sort((a, b) => a - b);
if (sorted.join(",") !== "7,19,42") throw new Error("bad sort");
console.log("OK, sorted:", sorted.join(" < "));
```

4. Check the output says `OK`.

YOU

Question 8: What are the key ideas?

CHATGPT

Answer 8: Key ideas

> Good retrieval matters more than a bigger model.

Cosine similarity is `\cos(\theta) = \frac{a \cdot b}{|a||b|}`. Read the Wikipedia page.

Use `top_k = 3` to start.

Quick check: what does RAG add?

○ Retrieval

○ Bigger GPUs

YOU

Question 9: Explain RAG simply.
Please keep it short.

CHATGPT

Answer 9: RAG in one minute

RAG = *retrieval* + generation. Steps:

1. Split documents into chunks.
2. Find chunks that match the question.
3. Give them to the model with the question.

Tiny Python version:

```
def retrieve(query, docs):
    """Return the docs that share a word with the query."""
    words = set(query.lower().split())
    results = []
    for doc in docs:
        if words & set(doc.lower().split()):
            results.append(doc)
```

```
    return results


docs = ["RAG uses retrieval", "Pizza is tasty", "retrieval helps <LLMs> & search"]
found = retrieve("Retrieval", docs)
assert len(found) == 2, found
print("OK, found:", found)
```

Run it with `python rag.py`.

YOU

Question 10: Compare search methods in a table.

CHATGPT

Answer 10: Comparison table

Here is how the options compare:

Method	Speed	Needs GPU?	Example
Keyword search	Fast	No	`grep "rag"`
Vector search	Medium	Sometimes	a | b pipes
Re-ranking	Slow	Yes	cross-encoder

Pick keyword search for small projects.

YOU

Question 11: Give me a study plan.
Please keep it short.

CHATGPT

Answer 11: Study plan

- Week 1
  - Python basics
  - Lists & dictionaries
- Week 2
  - APIs
  - Embeddings

3. Sort some scores in JavaScript:

```
const scores = [42, 7, 19];
const sorted = [...scores].sort((a, b) => a - b);
if (sorted.join(",") !== "7,19,42") throw new Error("bad sort");
console.log("OK, sorted:", sorted.join(" < "));
```

4. Check the output says `OK`.

YOU

Question 12: What are the key ideas?

CHATGPT

Answer 12: Key ideas

> Good retrieval matters more than a bigger model.

Cosine similarity is `\cos(\theta) = \frac{a \cdot b}{|a||b|}`. Read the Wikipedia page.

Use `top_k = 3` to start.

Quick check: what does RAG add?

○ Retrieval

○ Bigger GPUs

YOU

Question 13: Explain RAG simply.
Please keep it short.

CHATGPT

Answer 13: RAG in one minute

RAG = *retrieval* + generation. Steps:

1. Split documents into chunks.
2. Find chunks that match the question.
3. Give them to the model with the question.

Tiny Python version:

```
def retrieve(query, docs):
    """Return the docs that share a word with the query."""
    words = set(query.lower().split())
    results = []
    for doc in docs:
        if words & set(doc.lower().split()):
            results.append(doc)


    return results


docs = ["RAG uses retrieval", "Pizza is tasty", "retrieval helps <LLMs> & search"]
found = retrieve("Retrieval", docs)
assert len(found) == 2, found
print("OK, found:", found)
```

Run it with `python rag.py`.

YOU

Question 14: Compare search methods in a table.

CHATGPT

Answer 14: Comparison table

Here is how the options compare:

Method	Speed	Needs GPU?	Example
Keyword search	Fast	No	`grep "rag"`
Vector search	Medium	Sometimes	a | b pipes
Re-ranking	Slow	Yes	cross-encoder

Pick keyword search for small projects.

YOU

Question 15: Give me a study plan.
Please keep it short.

CHATGPT

Answer 15: Study plan

- Week 1
  - Python basics
  - Lists & dictionaries
- Week 2
  - APIs
  - Embeddings

3. Sort some scores in JavaScript:

```
const scores = [42, 7, 19];
const sorted = [...scores].sort((a, b) => a - b);
if (sorted.join(",") !== "7,19,42") throw new Error("bad sort");
console.log("OK, sorted:", sorted.join(" < "));
```

4. Check the output says `OK`.

YOU

Question 16: What are the key ideas?

CHATGPT

Answer 16: Key ideas

> Good retrieval matters more than a bigger model.

Cosine similarity is `\cos(\theta) = \frac{a \cdot b}{|a||b|}`. Read the Wikipedia page.

Use `top_k = 3` to start.

Quick check: what does RAG add?

○ Retrieval

○ Bigger GPUs

YOU

Question 17: Explain RAG simply.
Please keep it short.

CHATGPT

Answer 17: RAG in one minute

RAG = *retrieval* + generation. Steps:

1. Split documents into chunks.
2. Find chunks that match the question.
3. Give them to the model with the question.

Tiny Python version:

```
def retrieve(query, docs):
    """Return the docs that share a word with the query."""
    words = set(query.lower().split())
    results = []
    for doc in docs:
        if words & set(doc.lower().split()):
            results.append(doc)


    return results


docs = ["RAG uses retrieval", "Pizza is tasty", "retrieval helps <LLMs> & search"]
found = retrieve("Retrieval", docs)
assert len(found) == 2, found
print("OK, found:", found)
```

Run it with `python rag.py`.

YOU

Question 18: Compare search methods in a table.

CHATGPT

Answer 18: Comparison table

Here is how the options compare:

Method	Speed	Needs GPU?	Example
Keyword search	Fast	No	`grep "rag"`
Vector search	Medium	Sometimes	a | b pipes
Re-ranking	Slow	Yes	cross-encoder

Pick keyword search for small projects.

YOU

Question 19: Give me a study plan.
Please keep it short.

CHATGPT

Answer 19: Study plan

- Week 1
  - Python basics
  - Lists & dictionaries
- Week 2
  - APIs
  - Embeddings

3. Sort some scores in JavaScript:

```
const scores = [42, 7, 19];
const sorted = [...scores].sort((a, b) => a - b);
if (sorted.join(",") !== "7,19,42") throw new Error("bad sort");
console.log("OK, sorted:", sorted.join(" < "));
```

4. Check the output says `OK`.

YOU

Question 20: What are the key ideas?

CHATGPT

Answer 20: Key ideas

> Good retrieval matters more than a bigger model.

Cosine similarity is `\cos(\theta) = \frac{a \cdot b}{|a||b|}`. Read the Wikipedia page.

Use `top_k = 3` to start.

Quick check: what does RAG add?

○ Retrieval

○ Bigger GPUs

YOU

Question 21: Explain RAG simply.
Please keep it short.

CHATGPT

Answer 21: RAG in one minute

RAG = *retrieval* + generation. Steps:

1. Split documents into chunks.
2. Find chunks that match the question.
3. Give them to the model with the question.

Tiny Python version:

```
def retrieve(query, docs):
    """Return the docs that share a word with the query."""
    words = set(query.lower().split())
    results = []
    for doc in docs:
        if words & set(doc.lower().split()):
            results.append(doc)


    return results


docs = ["RAG uses retrieval", "Pizza is tasty", "retrieval helps <LLMs> & search"]
found = retrieve("Retrieval", docs)
assert len(found) == 2, found
print("OK, found:", found)
```

Run it with `python rag.py`.

YOU

Question 22: Compare search methods in a table.

CHATGPT

Answer 22: Comparison table

Here is how the options compare:

Method	Speed	Needs GPU?	Example
Keyword search	Fast	No	`grep "rag"`
Vector search	Medium	Sometimes	a | b pipes
Re-ranking	Slow	Yes	cross-encoder

Pick keyword search for small projects.

YOU

Question 23: Give me a study plan.
Please keep it short.

CHATGPT

Answer 23: Study plan

- Week 1
  - Python basics
  - Lists & dictionaries
- Week 2
  - APIs
  - Embeddings

3. Sort some scores in JavaScript:

```
const scores = [42, 7, 19];
const sorted = [...scores].sort((a, b) => a - b);
if (sorted.join(",") !== "7,19,42") throw new Error("bad sort");
console.log("OK, sorted:", sorted.join(" < "));
```

4. Check the output says `OK`.

YOU

Question 24: What are the key ideas?

CHATGPT

Answer 24: Key ideas

> Good retrieval matters more than a bigger model.

Cosine similarity is `\cos(\theta) = \frac{a \cdot b}{|a||b|}`. Read the Wikipedia page.

Use `top_k = 3` to start.

Quick check: what does RAG add?

○ Retrieval

○ Bigger GPUs

YOU

Question 25: Explain RAG simply.
Please keep it short.

CHATGPT

Answer 25: RAG in one minute

RAG = *retrieval* + generation. Steps:

1. Split documents into chunks.
2. Find chunks that match the question.
3. Give them to the model with the question.

Tiny Python version:

```
def retrieve(query, docs):
    """Return the docs that share a word with the query."""
    words = set(query.lower().split())
    results = []
    for doc in docs:
        if words & set(doc.lower().split()):
            results.append(doc)


    return results


docs = ["RAG uses retrieval", "Pizza is tasty", "retrieval helps <LLMs> & search"]
found = retrieve("Retrieval", docs)
assert len(found) == 2, found
print("OK, found:", found)
```

Run it with `python rag.py`.

YOU

Question 26: Compare search methods in a table.

CHATGPT

Answer 26: Comparison table

Here is how the options compare:

Method	Speed	Needs GPU?	Example
Keyword search	Fast	No	`grep "rag"`
Vector search	Medium	Sometimes	a | b pipes
Re-ranking	Slow	Yes	cross-encoder

Pick keyword search for small projects.

YOU

Question 27: Give me a study plan.
Please keep it short.

CHATGPT

Answer 27: Study plan

- Week 1
  - Python basics
  - Lists & dictionaries
- Week 2
  - APIs
  - Embeddings

3. Sort some scores in JavaScript:

```
const scores = [42, 7, 19];
const sorted = [...scores].sort((a, b) => a - b);
if (sorted.join(",") !== "7,19,42") throw new Error("bad sort");
console.log("OK, sorted:", sorted.join(" < "));
```

4. Check the output says `OK`.

YOU

Question 28: What are the key ideas?

CHATGPT

Answer 28: Key ideas

> Good retrieval matters more than a bigger model.

Cosine similarity is `\cos(\theta) = \frac{a \cdot b}{|a||b|}`. Read the Wikipedia page.

Use `top_k = 3` to start.

Quick check: what does RAG add?

○ Retrieval

○ Bigger GPUs

YOU

Question 29: Explain RAG simply.
Please keep it short.

CHATGPT

Answer 29: RAG in one minute

RAG = *retrieval* + generation. Steps:

1. Split documents into chunks.
2. Find chunks that match the question.
3. Give them to the model with the question.

Tiny Python version:

```
def retrieve(query, docs):
    """Return the docs that share a word with the query."""
    words = set(query.lower().split())
    results = []
    for doc in docs:
        if words & set(doc.lower().split()):
            results.append(doc)


    return results


docs = ["RAG uses retrieval", "Pizza is tasty", "retrieval helps <LLMs> & search"]
found = retrieve("Retrieval", docs)
```

```
assert len(found) == 2, found
print("OK, found:", found)
```

Run it with `python rag.py`.

YOU

Question 30: Compare search methods in a table.

CHATGPT

Answer 30: Comparison table

Here is how the options compare:

Method	Speed	Needs GPU?	Example
Keyword search	Fast	No	`grep "rag"`
Vector search	Medium	Sometimes	a | b pipes
Re-ranking	Slow	Yes	cross-encoder

Pick keyword search for small projects.